

Pipeline de Automação de Testes API — K6

K6 · Docker · GitHub Actions · Jenkins

Versão 1.0

Índice

1.	Visão Geral da Arquitetura	3
2.	Estrutura do Repositório	3
3.	Testes K6	4
4.	Endpoints Cobertos	6
5.	Docker	7
6.	GitHub Actions	8
7.	Jenkins Pipeline	9
8.	Referência Rápida de Comandos	10

1. Visão Geral da Arquitetura

Este documento descreve o pipeline completo de testes de performance e carga para a FakeStore API utilizando o K6. A arquitetura segue o princípio de separação de responsabilidades: o GitHub Actions atua como porteiro de qualidade (smoke test e geração da imagem Docker), enquanto o Jenkins é o executor que roda todos os testes em horários agendados — sempre a partir de uma imagem previamente aprovada.

git push	GitHub Actions	ghcr.io	Jenkins
Código local	Smoke Test + Push	Imagem aprovada	Pull + Executa todos

Regra chave: Se o smoke test falhar no GitHub Actions, a imagem Docker não é criada e o Jenkins nunca executa uma versão quebrada.

2. Estrutura do Repositório

O repositório contém a pasta k6/ com todos os testes e o Dockerfile, além dos arquivos de pipeline na raiz.

```
Teste_de_API_K6/  
  k6/  
    tests/  
      smoke-test.js  
      load-test.js  
      stress-test.js  
      spike-test.js  
    Dockerfile  
  docs/  
    k6.md  
  .github/workflows/ci.yml  
  Jenkinsfile  
  README.md
```

3. Testes K6

Sao 4 tipos de teste, cada um com objetivo especifico. Todos usam grupos (group()) para organizar os resultados por dominio de endpoints.

3.1 Smoke Test

Validacao minima para garantir que a API esta respondendo. Roda com 1 usuario virtual por 30 segundos. E o primeiro teste a ser executado — tanto no CI quanto no Jenkins.

```
export const options = { vus: 1, duration: '30s', thresholds: { http_req_duration: ['p(95)<1000'], http_req_failed: ['rate<0.01'], }, };
```

3.2 Load Test

Simula a carga normal esperada com 50 usuarios virtuais simultaneos por 30 segundos. Inclui metricas customizadas por grupo: product_duration, login_duration, cart_duration e requests_total.

```
export const options = { vus: 50, duration: '30s', thresholds: { http_req_duration: ['p(95)<500'], http_req_failed: ['rate<0.05'], product_duration: ['p(95)<400'], login_duration: ['p(95)<600'], }, };
```

3.3 Stress Test

Aumenta a carga gradualmente em etapas para identificar o ponto de saturacao da API. Util para descobrir o limite de capacidade antes de um lancamento.

Etapas	Duracao	VUs
Aquecimento	1 min	0 -> 50
Estavel	2 min	50
Subida	1 min	50 -> 100
Estavel	2 min	100
Pico	1 min	100 -> 200
Estavel	2 min	200
Resfriamento	1 min	200 -> 0

3.4 Spike Test

Simula um pico repentino de trafego como uma promocao ou evento viral. Sobe de 5 para 500 VUs em apenas 10 segundos.

Etapas	Duracao	VUs
Normal	10s	5
Spike	10s	5 -> 500

Pico mantido	1 min	500
Retorno	10s	500 -> 5
Recuperacao	30s	5
Finalizacao	10s	0

4. Endpoints Cobertos

Endpoint	Metodo	Smoke	Load	Stress	Spike
/products	GET	S	S	S	S
/products/:id	GET	S	S	S	
/products?limit=5	GET	S			
/products?sort=desc	GET	S			
/products/categories	GET	S			
/products/category/electronics	GET	S	S		
/carts	GET	S			
/carts/:id	GET	S	S	S	
/carts	POST	S	S	S	
/users	GET	S			
/users/:id	GET	S	S		
/auth/login	POST	S	S	S	S

S = coberto neste tipo de teste

5. Docker

O Dockerfile empacota o ambiente de testes completo garantindo execucao identica em qualquer maquina. A imagem e baseada na imagem oficial do Grafana K6.

5.1 Dockerfile

```
FROM grafana/k6:latest WORKDIR /tests COPY tests/ . ENTRYPOINT ["k6"] CMD ["run", "load-test.js"]
```

5.2 Comandos Docker

```
# Build da imagem docker build -t k6-fakestore ./k6 # Rodar teste especifico docker run --rm k6-fakestore run smoke-test.js docker run --rm k6-fakestore run load-test.js docker run --rm k6-fakestore run stress-test.js docker run --rm k6-fakestore run spike-test.js
```

5.3 Imagem no Registry (ghcr.io)

Apos cada push bem-sucedido na main, o GitHub Actions faz o push automatico da imagem para o GitHub Container Registry.

Item	Valor
URL da imagem	ghcr.io/erictonfelicidade86/teste_de_api_k6/k6:latest
Autenticacao	Automatica via GITHUB_TOKEN no Actions
Tag por commit	ghcr.io/.../k6:<SHA do commit>
Pull pelo Jenkins	docker pull ghcr.io/.../k6:latest

6. GitHub Actions

O arquivo `.github/workflows/ci.yml` define o pipeline de integracao continua. E disparado a cada push ou pull request na branch main.

6.1 Fluxo

Job	Acao	Condicao
test-k6	Executa smoke test	Sempre (push ou PR)
build-and-push	Build + push da imagem para ghcr.io	Somente push na main

6.2 ci.yml

```
name: CI - K6 Tests & Docker Push
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  test-k6:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: grafana/setup-k6-action@v1
      - run: k6 run k6/tests/smoke-test.js
  build-and-push:
    needs: test-k6
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: docker/login-action@v3
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/.../k6:latest
```


7. Jenkins Pipeline

O Jenkins executa todos os testes em sequencia via Jenkinsfile, agendado por cron (08h e 20h). Cada stage roda de forma independente — uma falha nao interrompe os demais.

7.1 Stages do Pipeline

Stage	Acao	Comportamento em falha
Checkout	git pull do Jenkinsfile	Para o pipeline
Docker Login	Login no ghcr.io com PAT	Para o pipeline
Pull Imagem K6	docker pull da imagem aprovada	Para o pipeline
Smoke Test	docker run smoke-test.js	Continua
Load Test	docker run load-test.js	Continua
Stress Test	docker run stress-test.js	Continua
Spike Test	docker run spike-test.js	Continua
Post Actions	docker logout + relatorio	Sempre executa

7.2 Configuracao de Credenciais

Em Manage Jenkins > Credentials, adicionar uma credencial do tipo Username with Password:

Campo	Valor
Kind	Username with password
Username	ErictonFelicidade86
Password	Personal Access Token (PAT) com escopo read:packages
ID	github-credentials

7.3 Fluxo de Decisao

Cenario	O que acontece	Resultado
Smoke passa no Actions	Build + push para ghcr.io	Imagem publicada
Smoke falha no Actions	Pipeline para — imagem NAO criada	Bloqueado
Jenkins executa no cron	Pull da imagem + todos os testes	Execucao agendada
Stage falha no Jenkins	catchError — proximo stage roda	Pipeline continua

8. Referencia Rapida de Comandos

Acao	Comando
Rodar smoke test local	<code>k6 run k6/tests/smoke-test.js</code>
Rodar load test local	<code>k6 run k6/tests/load-test.js</code>
Rodar stress test local	<code>k6 run k6/tests/stress-test.js</code>
Rodar spike test local	<code>k6 run k6/tests/spike-test.js</code>
Build da imagem Docker	<code>docker build -t k6-fakestore ./k6</code>
Rodar via Docker	<code>docker run --rm k6-fakestore run smoke-test.js</code>
Push manual para ghcr.io	<code>docker push ghcr.io/.../k6:latest</code>
Pull imagem no Jenkins	<code>docker pull ghcr.io/.../k6:latest</code>
Acessar Jenkins	<code>http://localhost:8080</code>
Ver logs no Jenkins	Jenkins > Job > Build #N > Console Output